ASSIGNMENT 2

FUNCTIONS AND OBJECT-ORIENTED PROGRAMMING IN C++

STUDENT DETAILS	
Name	Nirmal Kumar T
Register Number	1972260020
Programme	B.Tech – Computer Science and Engineering
Course	Object Oriented Programming with C++
Course Outcome	CO2
Total Marks	50
Assignment Nature	Individual

CO2 coverage: Function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, classes, member functions, access specifiers, arrays within a class, static members, arrays of objects and friend functions.

Important execution note: This report gives worked explanations, code solutions and expected results. The assignment requires each student to compile and execute every task personally. Add your own terminal screenshots, compiler warnings and timing measurements before submission.

Assignment Overview

The uploaded assignment contains Q1–Q6 and a Day-4 Design Challenge (Q7), for a total of 50 marks. Every task is assessed at Bloom's level K5 – Evaluate.

Question	Topic	Marks
Q1	Call by value, reference and pointer	6
Q2	Return by reference and its dangers	4
Q3	Default arguments and function overloading	6
Q4	Class with an array inside it	6
Q5	Static members and arrays of objects	7
Q6	Friend functions and inline functions	8
Q7	Diagnose, redesign and validate	13
TOTAL		50

Compilation procedure

Record your compiler version. The assignment gives g++ 13 as an example and recommends a C++17 build with warnings enabled.

```
g++ --version
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main
./main
```

On Windows, run main.exe after compilation. Keep one folder per question with the C++ source file, any input file and a screenshot of actual output.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Context: The hostel office applies merit (10%), sports (15%) and single-parent (25%) concessions. Compare the three parameter-passing methods.

A. Three implementations

```
#include <iostream>
using namespace std;

void byValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void byPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= *fee * percent / 100;
}
int main() {
    long long a=45000, b=45000, c=45000;
    byValue(a,10);
    byReference(b,10);
    byPointer(&c,10);
    cout << a << ' ' << b << ' ' << c << '\\n';
}
```

Expected output: 45000 40500 40500. A value parameter is a local copy. A reference is an alias to the caller's variable. A pointer stores its address; dereferencing it modifies the original variable.

B. Predict the supplied program

```
#include <iostream>
void change(int a, int& b, int* c) {
    a += 10; b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y=5, int z=10) {
    std::cout << x+y+z << ' ';
}
int main() {
    int a=1, b=2, c=3;
    change(a,b,&c);
    std::cout << a << ' ' << b << ' ' << c << '\\n';
    bigger(a,c)=100;
    std::cout << a << ' ' << c << '\\n';
    show(1); show(1,2); show(1,2,3);
    std::cout << '\\n';
}
```

Expected output:

```
1 12 13
1 100
16 13 6
```

After change(), a stays 1 because it is passed by value; b becomes 12 and c becomes 13. bigger(a,c) returns a reference to c, the larger value, so c becomes 100. show(1) uses y=5,z=10; show(1,2) uses z=10; show(1,2,3) uses no defaults.

C. Complete TODO 1

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
```

Expected supplied-program output: after call by value : 45000; after call by reference: 40500. Call by value changes only its local fee copy.

D. Alternative design and complexity

A return-value design is explicit: long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }. The caller must assign the result. All three calculations are O(1). Passing a long long by value copies one scalar; passing a large Student object by value copies the object, while reference and const reference pass an alias. Use const reference when the function only reads a large object. Use a mutable reference when in-place modification is part of the interface.

Q2 – Return by Reference and Its Dangers (4 Marks)

Context: The marks register stores marks in an array. The highest mark is to be returned by reference so +2 can be applied directly.

A. Safe array-element reference

```
#include <iostream>
using namespace std;

int& highest(int marks[], int n) {
    int best = 0;
    for (int i=1; i<n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

// Unsafe example: never use the returned reference.
int& badReference() {
    int local = 50;
    return local; // dangling reference
}

int main() {
    int marks[] = {72,91,85,91};
    highest(marks,4) += 2;
    for (int x : marks) cout << x << ' ';
    cout << '\\n';
}
```

Expected output: 72 93 85 91. The first maximum is updated. Returning a reference to an array element is valid while the array remains alive. Returning a reference to a local variable is invalid because its lifetime ends at function return; using the dangling reference is undefined behaviour. Compilers commonly warn about returning a reference to a local variable.

B. Reference, const reference and copy

int& lets the caller modify the referred element. const int& permits reading but not modification through that reference. Returning int gives a copy, so changing the copy does not change the array.

If the array is replaced by a growing std::vector, insertion can reallocate storage and invalidate existing references and pointers to its elements. An index is usually more robust across reallocation, provided the intended element still occupies that index.

C. Alternative return forms

```
int highestIndex(const int marks[], int n) {
    int best=0;
    for (int i=1; i<n; ++i)
        if (marks[i] > marks[best]) best=i;
    return best;
}
// Usage: marks[highestIndex(marks,n)] += 2;
```

Finding the highest mark is O(n) time and O(1) extra space whether returning an index, pointer or reference. An index avoids dangling storage addresses after vector reallocation, although insertions/erasures can change which logical item an index denotes. Capture a compiler warning for badReference(), but do not call or dereference it.

Q3 – Default Arguments and Function Overloading (6 Marks)

Context: Courier charge depends on weight, express delivery and insurance.

A. Explain the ambiguity

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express=false, bool insured=false);
```

charge(2.0) can match both the one-argument overload and the three-parameter overload using defaults. A two-argument call can also match the second and third declarations. The compiler reports an ambiguous call.

B. One unambiguous design

```
#include <iostream>
struct Parcel {
    double kilograms;
    bool express;
    bool insured;
};

double charge(double kg, bool express=false, bool insured=false) {
    double total=kg*50.0;
    if (express) total+=30.0;
    if (insured) total+=20.0;
    return total;
}
double charge(int grams) {
    return charge(grams/1000.0,false,false);
}
double charge(const Parcel& p) {
    return charge(p.kilograms,p.express,p.insured);
}
int main() {
    Parcel p{2.0,true,true};
    std::cout << charge(2) << '\\n';
    std::cout << charge(2.0) << '\\n';
    std::cout << charge(2.0f) << '\\n';
    std::cout << charge(2,true) << '\\n';
    std::cout << charge(2000) << '\\n';
    std::cout << charge(p) << '\\n';
}
```

Call selection: charge(2) selects int grams; charge(2.0) selects double kg; charge(2.0f) converts float to double; charge(2,true) selects the double overload; charge(2000) selects int grams; charge(p) selects Parcel.

Expected output:

```
50
100
100
130
100
150
```

C. Overload rules and alternative

The compiler prefers an exact match over a promotion, and a promotion over a standard conversion when comparing viable candidates. Default arguments fill omitted trailing parameters after overload selection. Defaults must be specified from the right so that omitted arguments are trailing arguments.

A single Parcel structure with optional fields is easier to extend with new options, but callers must populate a structure. A template is appropriate when the algorithm truly supports multiple types. Overload resolution occurs at compile time and adds no runtime selection cost. Charge calculation is O(1); passing Parcel by const reference avoids copying its fields.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Context: A SavingsAccount retains its last 10 transactions; older transactions are discarded.

A. Class and member functions

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
private:
    string accountNumber;
    long long balance;
    long long transactions[10]{};
    int count=0, next=0;
public:
    SavingsAccount(const string& number, long long opening)
        : accountNumber(number), balance(opening) {}
    void deposit(long long amount);
    bool withdraw(long long amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(long long amount) {
    if (amount<=0) return;
```

```
    balance+=amount;
    transactions[next]=amount;
    next=(next+1)%10;
    if (count<10) ++count;
}
bool SavingsAccount::withdraw(long long amount) {
    if (amount<=0 || amount>balance) return false;
    balance-=amount;
    transactions[next]=-amount;
    next=(next+1)%10;
    if (count<10) ++count;
    return true;
}
void SavingsAccount::miniStatement() const {
    int start=(count==10)?next:0;
    for (int i=0;i<count;++i)
        cout << transactions[(start+i)%10] << '\\n';
    cout << "Balance: " << balance << '\\n';
}
int main() {
    SavingsAccount a("SA1001",10000);
    a.deposit(500);
    a.withdraw(200);
    a.miniStatement();
}
```

The account number, balance, array and count are private to prevent outside code from corrupting account state. Public methods enforce valid deposits and withdrawals. Some member functions are declared in the class and defined outside it using SavingsAccount::.

B. Shifting versus circular index

Design	Insertion after full	Time	Space
Shift array	Move older entries left	O(k)	O(k)
Circular index	Overwrite oldest slot	O(1)	O(k)

For 50 transactions, change capacity to 50. A circular buffer requires only a capacity/index update and retains O(1) insertion. std::deque can also use push_back and pop_front. A fixed array uses bounded O(k) storage; shifting costs O(k) per insertion and circular insertion costs O(1). Printing k transactions costs O(k).

Q5 – Static Members and Arrays of Objects (7 Marks)

Context: A bus has a fixed number of seats. Ticket objects receive sequential seats; excess tickets are waitlisted.

A. Complete BusTicket code

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY=3;
    int seat;
    string name;
public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued<CAPACITY) seat=++issued;
    }
    static int count() { return issued; }
    void print() const {
        cout << name << ": "
             << (seat ? "Seat "+to_string(seat) : string("Waiting"))
             << '\\n';
    }
};
int BusTicket::issued=0;

int main() {
    BusTicket trip[4] = {BusTicket("Asha"),BusTicket("Bala"),
                         BusTicket("Chitra"),BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

TODO 1 checks issued against CAPACITY, increments issued and assigns the new value to seat. TODO 2 returns issued.

B. Predict Token output

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id=++next; }
    int get() const { return id; }
    static int issued() { return next-100; }
};
int Token::next=100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get()
              << ' ' << extra.get() << ' ' << Token::issued() << '\\n';
}
```

Expected output: 101 103 104 4. The counter begins at 100 and the constructor runs four times: once for each of three array elements and once for extra.

C. Static member explanation and alternatives

The static issued member belongs to the class and exists once for all objects. A static member function has no this pointer and cannot directly access non-static members such as seat or name. The static data member is defined outside the class with int BusTicket::issued = 0;.

If tickets are created in a function called twice, the static count continues; it is not reset when the function returns. A global counter also takes O(1) space and updates in O(1), but weakens encapsulation. A TicketOffice object passed to tickets supports separate counters for multiple buses and is often better when multiple bus instances must operate independently.

Q6 – Friend Functions and Inline Functions (8 Marks)

Context: A Train Journey Planner compares two Time objects with private hour and minute values.

A. Complete TODO 1 and TODO 2

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
    int h,m;
public:
    Time(int hh,int mm):h(hh),m(mm){}
    friend int minutesBetween(const Time& a,const Time& b);
    friend bool earlier(const Time& a,const Time& b);
};
int minutesBetween(const Time& a,const Time& b) {
    int first=a.h*60+a.m;
    int second=b.h*60+b.m;
    return abs(first-second);
}
bool earlier(const Time& a,const Time& b) {
    if (a.h!=b.h) return a.h<b.h;
    return a.m<b.m;
}
int main() {
    Time departure(9,45), arrival(13,10);
    cout << minutesBetween(departure,arrival) << ' '
         << boolalpha << earlier(arrival,departure) << '\\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight; their difference is 205. Arrival is not earlier than departure.

B. Friend, member and getter alternatives

A friend function is not a member, but the class grants it access to private fields. It is called as minutesBetween(a,b) and can treat both operands symmetrically. A member version is called as a.minutesTo(b). Making fields public weakens encapsulation and allows callers to bypass class invariants.

```
class Time {
    int h,m;
public:
    Time(int hh,int mm):h(hh),m(mm){}
    int minutesTo(const Time& other) const {
        return (other.h*60+other.m)-(h*60+m);
    }
    bool isEarlierThan(const Time& other) const {
        return h<other.h || (h==other.h && m<other.m);
    }
};
```

The inline keyword suggests that a function may be expanded at its call site and permits suitable definitions in multiple translation units; it does not force the compiler to inline. Modern optimization settings can inline functions even without the keyword.

C. Correctness tests

Test	Expected
09:45 and 13:10	205 minutes
Same time twice	0 minutes; earlier is false
11:55 and 12:05	10 minutes; 11:55 is earlier
12:05 and 11:55	10 minutes; 12:05 is not earlier

D. Required timing investigation

The assignment requires timing 10,000,000 calls with and without inline, compiled with -O0 and -O2. Use std::chrono::steady_clock, repeat each build several times and report the median. Do not invent timing values: they depend on the compiler and machine.

```
g++ -std=c++17 -O0 timing.cpp -o timing_O0
g++ -std=c++17 -O2 timing.cpp -o timing_O2
./timing_O0
./timing_O2
```

The operation per call is O(1); ten million calls take O(n) total time. Use equivalent test programs for inline and non-inline versions and record all four timings. The same-time and either-side-of-noon tests should be verified manually.

Q7 – Diagnose, Redesign and Validate: Day-4 Challenge (13 Marks)

The assignment requires selecting one Q1–Q6 program with a limitation, diagnosing it using evidence, setting a measurable objective, redesigning, predicting, implementing, validating and defending the final design. This worked example selects Q4's transaction history.

A. Diagnose the limitation – 2 marks

When a fixed array is full, a shifting implementation moves older entries left before appending the new transaction. With k retained entries, an insertion can require O(k) element moves. The evidence should include the student's own implementation and measurement.

B. Define the objective – 2 marks

Retain the latest 50 transactions and reduce insertion work from O(k) element moves to O(1), while keeping storage bounded at O(50).

C. Revised circular-buffer design – 4 marks

```
class TransactionHistory {
    static constexpr int CAPACITY=50;
    long long values[CAPACITY]{};
    int count=0, next=0;
public:
    void add(long long amount) {
        values[next]=amount;
        next=(next+1)%CAPACITY;
        if (count<CAPACITY) ++count;
    }
    int size() const { return count; }
    long long oldestFirst(int position) const {
        int start=(count==CAPACITY)?next:0;
        return values[(start+position)%CAPACITY];
    }
};
```

The next index identifies where the next transaction is written. Once full, each new value overwrites the oldest value. The design uses O(50) storage and O(1) insertion. Its trade-off is more careful indexing when displaying the records.

D. Prediction – 1 mark

After the buffer is full, each insertion performs one array assignment, one modulo/index update and a bounded count check. The predicted insertion complexity is O(1), rather than O(k) for shifting.

E. Implement and validate – 2 marks

Input sequence	Expected retained history
Insert 1 to 10	1 through 10
Insert 11	2 through 11

Input sequence	Expected retained history
Insert 1 through 50	1 through 50
Insert 51	2 through 51
Insert 100	51 through 100

Run both original and redesigned versions with the same input sequence and number of insertions. Measure with steady_clock and repeat runs. Fill this table with your actual results:

Version	Capacity	Insertions	Measured time
Shifting array	50	10,000,000	Your measured value
Circular buffer	50	10,000,000	Your measured value

F. Defend the final design – 2 marks

The circular buffer achieves O(1) insertion with bounded O(50) storage. It is suitable for a fixed-size rolling history. A shifting array may remain preferable when the history is tiny and simplicity matters more than insertion cost. Use actual measurements to state whether a practical speedup occurred.

Final Conclusion

The assignment applies C++ function parameter passing, safe reference returns, overload resolution, class design, static members, friend functions and inline functions. The Day-4 challenge demonstrates a redesign that reduces insertion complexity for a bounded transaction history.

Submission Checklist

• Create repository DSA01-1972260020-Assignment2.

• Create Task01 through Task07 folders.

• Place source files, input files and output screenshots in the relevant task folders.

• Add README.md with name, register number, compiler/version, exact compile command, run steps and a short result note for every task.

• Commit progressively, at least one commit per task.

• Add your working GitHub repository link to the report cover page and submit it.

• Include actual predictions written before execution, observed outputs, compiler diagnostics and timing measurements.

Academic evidence reminder: This is a worked solution guide based on the uploaded Assignment 2 brief. You must execute each program yourself and replace all evidence placeholders with your own screenshots, compiler messages and measurements before submission.

APPENDIX – ACTUAL PROGRAM EXECUTION SCREENSHOTS

Evidence generated by compiling and running the assignment examples in a local C++17 environment using g++ with -Wall -Wextra -pedantic. These are genuine execution captures from this environment, not screenshots from the student's own computer.

Q1 – Execution evidence

TERMINAL / Q1

```
Actual local compile and run; source based on the worked program in the assignment PDF.

$ g++ -std=c++17 -Wall -Wextra -pedantic Q1.cpp -o Q1
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q1
Program output:
45000 40500 40500
1 12 13
1 100
16 13 6
```

Q2 – Execution evidence

TERMINAL / Q2

Actual local compile and run; source based on the worked program in the assignment PDF.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q2.cpp -o Q2
Compilation: SUCCESS (exit code 0)
Compiler diagnostic:       |                          ^~~~~
$ ./Q2
Program output:
72 93 85 91
```

Q3 – Execution evidence

TERMINAL / Q3

Actual run. Note: charge(2) returns 0.1 because the int overload treats 2 as grams.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q3.cpp -o Q3
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q3
Program output:
0.1
100
100
130
100
150
```

Important source inconsistency: The assignment PDF lists the expected first output for charge(2) as 50, but its supplied overload treats the integer argument as grams and divides by 1000. Therefore the code actually prints 0.1. The other outputs match: 100, 100, 130, 100, 150. Confirm with your instructor whether the overload or expected output should be changed.

Q4 – Execution evidence

TERMINAL / Q4

Actual local compile and run; source based on the worked program in the assignment PDF.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q4.cpp -o Q4
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q4
Program output:
500
-200
Balance: 10300
```

Q5 – Execution evidence

TERMINAL / Q5

Actual local compile and run; source based on the worked program in the assignment PDF.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q5.cpp -o Q5
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q5
Program output:
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
101 103 104 4
```

Q6 – Execution evidence

TERMINAL / Q6

Actual local compile and run; source based on the worked program in the assignment PDF.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q6.cpp -o Q6
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q6
Program output:
205 false
0 false
10 true
10 false
```

Q7 – Execution evidence

TERMINAL / Q7

Actual local compile and run; source based on the worked program in the assignment PDF.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic Q7.cpp -o Q7
Compilation: SUCCESS (exit code 0)
Compiler warnings: none
$ ./Q7
Program output:
Retained count: 50
First: 51 Last: 100
```

Q2 WARNING – Execution evidence

TERMINAL / Q2_WARNING

Compiler diagnostic demonstrating the dangling-reference risk.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic -Wreturn-local-addr -c Q2.cpp
Compiler warning captured:
warning: reference to local variable ‘local’ returned
note: declared here
The unsafe function was compiled only; it was NOT called.
```

timing O0 – Execution evidence

TERMINAL / timing_O0

```
Actual local timing run; 10 million calls; median of five runs.

$ g++ -std=c++17 -O0 timing.cpp -o timing_O0
$ ./timing_O0
10,000,000 calls; median of 5 runs (microseconds)
inline: 28816
noinline: 32171
```

Timing values are specific to this environment. They are not measurements from the student's own PC. O0 median: inline 28,816 μs; noinline 32,171 μs.

timing O2 – Execution evidence

TERMINAL / timing_O2

```
Actual local timing run; 10 million calls; median of five runs.

$ g++ -std=c++17 -O2 timing.cpp -o timing_O2
$ ./timing_O2
10,000,000 calls; median of 5 runs (microseconds)
inline: 0
noinline: 18969
```

O2 median: inline 0 μs (optimized away / effectively below timer resolution); noinline 18,969 μs. This illustrates that optimizer behavior affects the comparison; do not present these as universal performance claims.

Q7 BENCHMARK – Execution evidence

TERMINAL / Q7_BENCHMARK

```
Actual local benchmark; one million insertions; capacity 50; median of five runs.

$ g++ -std=c++17 -O2 q7bench.cpp -o q7bench
$ ./q7bench
1,000,000 insertions; capacity 50; median of 5 runs (microseconds)
shift array: 18838
circular buffer: 5346
```

This is a local illustrative benchmark of 1,000,000 insertions at capacity 50. Results depend on hardware and compiler; rerun on the submission computer if the assignment requires personal-machine timings.

Submission notes

1. The screenshots above verify that these example programs compiled and ran in this environment. 2. Replace or supplement them with screenshots from your own IDE/terminal if your instructor requires personal execution evidence. 3. Q2 warning evidence was generated by compiling the unsafe function only; the dangling reference was never called. 4. Q6 and Q7 timings are environment-dependent. 5. Q3 contains the mismatch explained above; seek clarification before final submission.